

A Late Acceptance Hill-Climbing Algorithm for Flexible Job Shop Scheduling Problems with Worker Flexibility

Eduard Iorga
ioed25ss@student.ju.se
Jönköping University
Jönköping, Sweden

Ben Bekir Ertugrul
erbe25sf@student.ju.se
Jönköping University
Jönköping, Sweden

Alexandros Tzanetos
alexandros.tzanetos@ju.se
Jönköping University
Jönköping, Sweden

Abstract

The Flexible Job Shop Scheduling Problem with Worker constraints (FJSSP-W) is a challenging optimization problem that simultaneously coordinates machine assignment, worker allocation, and operation sequencing. This paper proposes a Late Acceptance Hill Climbing (LAHC) metaheuristic to address the FJSSP-W. LAHC uses a single-state trajectory with a history-based acceptance criterion, enabling straightforward parameter tuning and interpretation. Our implementation explores a neighborhood structure comprising machine and worker reassignments, as well as job-sequence permutations, while using a secondary objective of workload balance to break ties in makespan values. Experimental results on the competition's benchmark demonstrate that the proposed LAHC approach is a promising algorithm to address the problem.

CCS Concepts

• Computing methodologies → Search methodologies; • Applied computing → Operations research.

Keywords

Flexible Job Shop Scheduling, Worker Flexibility, Metaheuristics

ACM Reference Format:

Eduard Iorga, Ben Bekir Ertugrul, and Alexandros Tzanetos. 2026. A Late Acceptance Hill-Climbing Algorithm for Flexible Job Shop Scheduling Problems with Worker Flexibility. In *Genetic and Evolutionary Computation Conference (GECCO Companion '26)*, July 13–17, 2026, San Jose, Costa Rica. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3795101.3815569>

1 Introduction

In FJSSP-W, a set of jobs must be allocated on a set of machines to minimize the makespan. Additionally, each operation should be assigned both a compatible machine and a qualified worker capable of operating it [4].

To navigate this expanded search space, we implemented Late Acceptance Hill Climbing (LAHC), a trajectory-based metaheuristic that builds on traditional hill climbers, with a history buffer H of length L [2]. Unlike steepest-descent methods that often converge prematurely, LAHC is uniquely motivated by its ability to accept non-improving moves based on historical performance [1]. This mechanism, paired with our implemented neighborhood operators, enables the search to escape local optima by comparing a candidate

solution not only to the current state but also to a state from L iterations ago. This provides an adaptive acceptance criterion while maintaining minimal parameter dependency.

2 Late Acceptance Hill-Climbing Algorithm

Algorithm 1 Late Acceptance Hill-Climbing

Input: FJSSP-W instance I , history length L , max iterations N

Output: The best solution found S^*

```
1: Generate a random valid solution  $S$ 
2:  $S^* \leftarrow S$ 
3:  $H \leftarrow \{mksp(S)\}^L$  // Initialize history buffer
4:  $i \leftarrow 0$ 
5: while  $i < N$  do
6:   Neighborhood Operator Selection( $S, p_{mut}, p_{swap}$ )
7:    $v \leftarrow i \pmod L$  // Pick an element to compare with
8:   if  $mksp(S') < mksp(S) \vee mksp(S') < H[v]$  then
9:      $S \leftarrow S'$ 
10:  else if  $mksp(S') = mksp(S) \wedge wb(S') < wb(S)$  then
11:     $S \leftarrow S'$ 
12:  end if
13:  if  $S$  is better than  $S^*$  then
14:     $S^* \leftarrow S$ 
15:  end if
16:   $H[v] \leftarrow mksp(S); i \leftarrow i + 1$ 
17: end while
18: return  $S^*$ 
```

The proposed approach initializes with a random solution, in which operations are iteratively scheduled by assigning a random valid job to an available machine-worker pair. This solution's makespan is used to populate the history buffer H , establishing a baseline for the next L iterations. From this point, it explores the search space over N iterations, as described in Algorithm 1, using a combination of three neighborhood operators defined in Algorithm 2. The operators were designed to ensure reachability across the solution space and efficient convergence using a redundant sequence-based strategy.

Machine and Worker Mutation: Re-assign an operation to a new machine-worker pair to reduce machine-specific bottlenecks and optimize workload balance.

Job Priority Swapping: Exchange two positions in the operation sequence in order to fine-tune the sequence.

Job Priority Shifting: Shifts an operation to a new position in the operation sequence. Allows for large-scale reorganization, which would require multiple swap moves.



This work is licensed under a Creative Commons Attribution 4.0 International License. *GECCO Companion '26, San Jose, Costa Rica*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2488-6/2026/07
<https://doi.org/10.1145/3795101.3815569>

Algorithm 2 Neighborhood Operator Selection

Input: Current solution S ,
mutation probability p_{mut} , swapping probability p_{swap}
Output: Neighbor solution S'

```

1:  $r \sim U(0, 1)$  // Generate a uniform random number
2: if  $r < p_{mut}$  then
3:   Assign operation  $o$  to new valid machine  $m$  and worker  $w$ 
4: else if  $r < p_{mut} + p_{swap}$  then
5:   Swap job indices at positions  $pos_1$  and  $pos_2$ 
6: else
7:   Move job index from  $pos_{src}$  to  $pos_{dest}$ 
8: end if
9: return  $S'$ 

```

A hierarchical acceptance criterion is employed, in which a candidate is accepted if it yields a better makespan (mks_p) than either the current solution or the solution recorded L iterations earlier. In cases of a tie in makespan, the algorithm evaluates the workload balance (wb) as a secondary objective.

We follow the official solution representation, i.e., a set of three vectors, comprising an operation sequence, a machine assignment vector, and a worker assignment vector. We also use the library's decoding procedure [4].

Our implementation and the experimental results are available at GitHub¹.

3 Experiment setup

To evaluate the performance of the proposed LAHC, a systematic benchmarking procedure was implemented in accordance with the official competition guidelines. Each problem instance was run 10 times to ensure experimental significance. For fitness evaluation, we used the standard makespan objective and the workload balance functions provided by the competition library².

Our LAHC implementation uses a history length of $L = 3000$ and stops at $N = 300,000$. As a result, we use a budget of 300,000, which is significantly less than the fixed function evaluation budget of 5,000,000 requested from the competition, but addresses the lack of computational resources. The neighborhood selection probabilities for Algorithm 2 were set to $p_{mut} = 0.4$ and $p_{swap} = 0.4$ after some preliminary tests.

The experiments were conducted on a server with an AMD Ryzen 9 9950X 16-core CPU and 192 GB of RAM running Python 3.10.

4 Results

Table 1 summarizes the performance of LAHC across all instances and compares it with a lower bound (LB) found by a Genetic Algorithm (GA) [3]. The algorithm achieves the lower bound on 0_BG_42 and 2b_He_1. In larger instances like 2a_Hs_61 and 2d_Hv_18, the solver found solutions significantly lower than the provided reference LB. With the exception of 2a_Hs_54, low standard deviations indicate stable convergence patterns.

Table 1: LAHC results against benchmark lower bounds

Instance	Reference	LAHC		
	LB	Best	Median	Std
0_BG_42	80.0	80	85.0	2.79
0_BG_46	110.0	126	134.5	5.36
0_BG_60	491.0	737	751.0	8.3
1_Br_12	479.0	521	530.5	7.06
1_Br_14	640.0	698	708.5	5.38
1_Br_7	143.0	160	174.0	8.25
2a_Hs_18	1179.0	1189	1219.5	20.09
2a_Hs_1	52.0	53	56.5	1.55
2a_Hs_38	2005.0	2256	2277.0	19.66
2a_Hs_40	1640.0	1618	1642.5	21.19
2a_Hs_54	7954.0	8048	8602.0	387.43
2a_Hs_61	1468.0	886	913.5	19.1
2a_Hs_63	605.0	391	397.0	6.73
2b_He_1	51.0	51	53.0	0.95
2b_He_6	601.0	545	561.5	16.72
2c_Hr_28	909.0	1050	1073.5	10.07
2c_Hr_38	1780.0	2154	2191.0	20.19
2c_Hr_50	5791.0	5820	5894.5	53.9
2d_Hv_18	1827.0	1104	1117.0	10.24
2d_Hv_30	1178.0	1485	1518.5	16.21
2d_Hv_5	580.0	523	537.0	11.97
3_DP_15	2328.0	3432	3488.0	24.72
3_DP_18	2305.0	3430	3463.5	32.4
3_DP_1	2728.0	3002	3055.0	33.01
3_DP_9	2270.0	3080	3129.5	23.69
4_CB_10	949.0	1014	1034.0	11.91
5_K_3	7.0	9	10.5	1.16
5_K_4	11.0	20	22.5	3.03
6_F_14	538.0	554	564.0	17.22
6_F_20	1208.0	1245	1304.0	22.76

Overall, these results suggest that while performance varies across instance categories, the proposed LAHC provides a reliable and computationally lean framework for FJSSP-W, particularly effective for the Hs, He, and Hv datasets.

References

- [1] Mohamed Elamine Athmani, Taha Arbaoui, Younes Mimene, and Farouk Yalaoui. 2022. Efficient heuristics and metaheuristics for the unrelated parallel machine scheduling problem with release dates and setup times. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Boston, Massachusetts) (GECCO '22). Association for Computing Machinery, New York, NY, USA, 177–185. doi:10.1145/3512290.3528857
- [2] Edmund K. Burke and Yuri Bykov. 2017. The late acceptance Hill-Climbing heuristic. *European Journal of Operational Research* 258, 1 (2017), 70–78. doi:10.1016/j.ejor.2016.07.012
- [3] David Hutter, Michael Hellwig, and Thomas Steinberger. 2024. An Interior-point Genetic Algorithm with Restarts for Flexible Job Shop Scheduling Problems. In *2024 IEEE Congress on Evolutionary Computation (CEC)*. 01–09. doi:10.1109/CEC60901.2024.10611934
- [4] David Hutter, Thomas Steinberger, and Michael Hellwig. 2025. A Benchmarking Environment for Worker Flexibility in Flexible Job Shop Scheduling Problems. arXiv:2501.16159 [cs.NE] <https://arxiv.org/abs/2501.16159>

¹<https://github.com/jrc-rodec/FJSSP-W-Competition/issues/9>

²<https://github.com/jrc-rodec/FJSSP-W-Competition/blob/main/util/evaluation.py>